SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Scenario-Based Task (High)
Assessment Tool Weightage: 35%

Dr.G.Ayyappan

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Given the scenario of a School Management System, identify relations and write SQL to list all students with their class teacher and grade.	BL3 – Apply	**5**
2	A hospital wants to track patient appointments. Design the relational schema and write SQL to find doctors with more than 10 appointments this month.	BL3 – Apply	**5**
3	In an e-commerce scenario, write SQL queries using sub-queries to find products that have never been ordered.	BL3 – Apply	**5**
4	For a university database, write relational algebra expressions to find students enrolled in both 'DBMS' and 'OS' courses.	BL3 – Apply	**5**
5	Given a payroll scenario, create a view showing employees whose salary is below department average and write a query to update their salary by 10%.	BL6 – Create	**5**
6	In a banking scenario, apply JOIN operations to retrieve customer account details, transaction history, and branch information.	BL3 – Apply	**5**

7	A retail store needs daily sales summary. Write SQL with GROUP BY and HAVING to report total sales per category exceeding Rs. 10,000.	BL3 – Apply	**5**
8	For a library system scenario, construct tuple relational calculus expressions to find members who borrowed books from all genres.	BL3 – Apply	**5**
9	Given an inventory management scenario, define CHECK and FOREIGN KEY constraints and demonstrate their enforcement through SQL.	BL3 – Apply	**5**
10	Design a relational schema for an HR system and create materialized views for monthly attendance and performance summaries.	BL6 – Create	**5**

Code of Conduct:

I V. Sujithra (192521445) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Scenario based assessment					
Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts

		address the scenario			
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1. Debug the Incorrect JOIN Query

Given Query

SELECT s.Name, d.DeptName

FROM Student s, Department d;

Error

The query uses a **Cartesian product** because there is no condition connecting Student and Department.

So, every student is combined with every department.

For example, if there are **9 students and 3 departments**, the query can produce:

rows instead of 9 correct student-department combinations.

The intention is to display each student's own department name. The relationship is based on DeptID.

Correct Query

SELECT s.Name, d.DeptName

FROM Student s

INNER JOIN Department d

ON s.DeptID = d.DeptID;

Explanation

- Student s gives the student table an alias s.
- Department d gives the department table an alias d.
- ON s.DeptID = d.DeptID connects the two tables.
- INNER JOIN returns only matching department records.

Expected Result

Name	**DeptName**
Aravind	Computer Science
Bhavya	Computer Science
Chandran	Computer Science
Deepak	Electronics

Esha	Electronics
Farhan	Electronics
Gowri	Mechanical
Hari	Mechanical
Indra	Mechanical

Conclusion

The error was the **missing JOIN condition**. Using INNER JOIN ... ON s.DeptID = d.DeptID removes the unwanted Cartesian product and returns the correct department for each student. The PDF explicitly identifies the original query's intention as listing each student with their own department name.

Q2. Debug the Correlated Subquery

Given Query

```
SELECT s.Name, s.DeptID, s.Marks
FROM Student s
WHERE s.Marks > (SELECT AVG(Marks) FROM Student);
```

Error

The inner query:

```
SELECT AVG(Marks) FROM Student
```

calculates the **average marks of all students**.

But the requirement is to find students who scored above the **average marks of their own department**.

Therefore, the subquery must be correlated with the outer query using DeptID.

Correct Query

```
SELECT s.Name, s.DeptID, s.Marks
FROM Student s
WHERE s.Marks >
(
    SELECT AVG(s2.Marks)
```

```
    FROM Student s2
    WHERE s2.DeptID = s.DeptID
);
```

How it Works

For every student:

1. The outer query selects one student.
2. The inner query finds students belonging to the same department.
3. AVG(s2.Marks) calculates that department's average.
4. The student's marks are compared with that average.
5. Only students scoring above their department average are returned.

Example

For Department 1:

Marks:

- Aravind = 85
- Bhavya = 78
- Chandran = 90

Average:

So:

- Aravind → 85 > 84.33 ✓
- Bhavya → 78 > 84.33 ✗
- Chandran → 90 > 84.33 ✓

Conclusion

The original query calculates the **overall average**, whereas the corrected correlated subquery calculates the **average of each student's own department**.

Q3. Debug and Optimize the View

Given Query

```
CREATE VIEW DeptAvgMarks AS
SELECT Name, DeptID, AVG(Marks) AS AvgMarks
```

```
FROM Student
GROUP BY DeptID;


SELECT * FROM DeptAvgMarks;
```

Error

The query selects:

Name, DeptID, AVG(Marks)

but only groups by:

GROUP BY DeptID

Name is neither:

- included in GROUP BY, nor
- used inside an aggregate function.

Therefore, in MySQL with ONLY_FULL_GROUP_BY enabled, this produces an error.

Also, logically, if we are calculating **department average marks**, individual student Name should not be included.

Correct and Optimized View

```
CREATE VIEW DeptAvgMarks AS
SELECT DeptID,
    AVG(Marks) AS AvgMarks
FROM Student
GROUP BY DeptID;
```

Query the View

```
SELECT *
FROM DeptAvgMarks;
```

Better Version with Department Name

Since the database contains a Department table, we can display the department name also:

```
CREATE VIEW DeptAvgMarks AS
SELECT d.DeptID,
    d.DeptName,
```

AVG(s.Marks) AS AvgMarks

FROM Department d

JOIN Student s

ON d.DeptID = s.DeptID

GROUP BY d.DeptID, d.DeptName;

Then:

SELECT *

FROM DeptAvgMarks;

Expected Result

DeptID	DeptName	AvgMarks
1	Computer Science	84.33
2	Electronics	70.00
3	Mechanical	51.00

Conclusion

The main error is the use of Name without grouping it. Since the purpose is department-wise average marks, the corrected view groups by DeptID and calculates AVG(Marks). The original view definition and its error scenario are given in the assessment.

Q4. Debug the Relational Algebra Expression

Given Expression

The intention is to perform a **natural join** between Student and Department using DeptID.

Error

The projection is performed **before** the join:

This removes DeptID from the Student relation.

But DeptID is the common attribute required for the natural join.

Therefore, the join cannot correctly match students with departments.

Correct Expression

First perform the natural join:

Then perform projection:

Step-by-Step

Step 1 – Natural Join

The common attribute is:

DeptID

So Student records are matched with their corresponding Department records.

Step 2 – Projection

After joining, select only:

Name

Age

Therefore:

Example Result

Name	**Age**
Aravind	20
Bhavya	21
Chandran	20
Deepak	22
Esha	21
Farhan	22
Gowri	20
Hari	21
Indra	20

Conclusion

The error is caused by **projecting DeptID away before performing the join**. The join should be performed first, followed by projection.

Q5. Referential Integrity Violation

Given Student Table

CREATE TABLE Student (

```
    RollNo INT PRIMARY KEY,
    Name VARCHAR(50),
    DeptID INT,
    Age INT,
    Marks INT
);
```

The Department table contains only:

DeptID = 1

DeptID = 2

DeptID = 3

But the following record is inserted:

```
INSERT INTO Student
(RollNo, Name, DeptID, Age, Marks)
VALUES
(110, 'Arun', 9, 20, 78);
```

Error

DeptID = 9 does not exist in the Department table.

Therefore, this violates **referential integrity**.

A student's DeptID should refer to an existing department.

The assessment specifically gives DeptID = 9 as the invalid value while the Department table has only 1, 2 and 3.

Correct Table Definition

First create the Department table:

```
CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50),
    HOD VARCHAR(50)
);
```

Then create Student with a foreign key:

```
CREATE TABLE Student (
   RollNo INT PRIMARY KEY,
   Name VARCHAR(50),
   DeptID INT,
   Age INT,
   Marks INT,
   FOREIGN KEY (DeptID)
      REFERENCES Department(DeptID)
);
```

Correct Insert

Since valid department IDs are 1, 2 and 3:

```
INSERT INTO Student
(RollNo, Name, DeptID, Age, Marks)
VALUES
(110, 'Arun', 1, 20, 78);
```

Invalid Insert

```
INSERT INTO Student
VALUES (111, 'Kumar', 9, 20, 78);
```

This will be rejected because Department 9 does not exist.

Conclusion

A **FOREIGN KEY constraint** should be used to enforce referential integrity between Student and Department.

Q6. Optimize the Nested Subquery Using JOIN

Given Query

```
SELECT s.Name,
      (SELECT d.DeptName
       FROM Department d
```

```
    WHERE d.DeptID = s.DeptID) AS DeptName
FROM Student s;
```

The query uses a correlated scalar subquery to find the department name for each student. The assessment asks to rewrite this using a JOIN for better efficiency.

Optimized Query

```
SELECT s.Name,
    d.DeptName
FROM Student s
INNER JOIN Department d
ON s.DeptID = d.DeptID;
```

Explanation

In the original query, the department lookup is written as a subquery for each student.

The optimized query directly joins the two tables using:

ON s.DeptID = d.DeptID

This makes the relationship explicit and generally gives the database optimizer a better opportunity to use indexes and efficient join strategies.

Example Result

Name	**DeptName**
Aravind	Computer Science
Bhavya	Computer Science
Chandran	Computer Science
Deepak	Electronics
Esha	Electronics
Farhan	Electronics
Gowri	Mechanical
Hari	Mechanical
Indra	Mechanical

Conclusion

Replacing the correlated lookup with an INNER JOIN makes the query simpler and more suitable for optimization.

Q7. Debug the GROUP BY Query

Given Query

SELECT DeptID, AVG(Marks) AS AvgMarks

FROM Student

GROUP BY RollNo;

Error

The query selects:

DeptID

AVG(Marks)

but groups the records by:

RollNo

RollNo identifies an individual student. Therefore, the query calculates an average for each student rather than an average for each department.

The question requires **department-wise average marks**.

The assessment identifies the incorrect GROUP BY RollNo as the source of the wrong aggregation.

Correct Query

SELECT DeptID,

AVG(Marks) AS AvgMarks

FROM Student

GROUP BY DeptID;

Expected Result

DeptID	AvgMarks
1	84.33
2	70.00

3	51.00

Explanation

For Department 1:

For Department 2:

For Department 3:

Conclusion

Since the requirement is department-wise aggregation, DeptID must be used in the GROUP BY clause.

Q8. Debug the UPDATE Statement

Given Query

UPDATE Student

SET Marks = Marks + 5;

Error

The query does not contain a WHERE clause.

Therefore, **all students** in the Student table receive 5 additional marks.

But the intention is to give bonus marks only to students in **Department 2**.

Correct Query

UPDATE Student

SET Marks = Marks + 5

WHERE DeptID = 2;

Explanation

The condition:

WHERE DeptID = 2

restricts the update to students belonging to Department 2.

From the sample data, Department 2 students are:

Name	DeptID	Original Marks
Deepak	2	60

Esha	2	55
Farhan	2	95

After update:

Name	**New Marks**
Deepak	65
Esha	60
Farhan	100

Important Point

Always use a suitable WHERE condition when updating selected rows.

Without:

WHERE DeptID = 2

the statement modifies the entire table.

Conclusion

The logical error is the **missing WHERE clause**. Adding WHERE DeptID = 2 ensures that only Department 2 students receive the bonus.

Q9. Optimize the Multi-Table SQL Query

Given Query

```
SELECT s.Name, s.Marks,
      (SELECT DeptName
       FROM Department
       WHERE DeptID = s.DeptID) AS DeptName,
      (SELECT HOD
       FROM Department
       WHERE DeptID = s.DeptID) AS HeadOfDept
FROM Student s
WHERE s.DeptID IN (
   SELECT DeptID
```

```
    FROM Department
    WHERE DeptName IN
    ('Computer Science', 'Electronics')
);
```

The query uses multiple subqueries to retrieve DeptName, HOD, and department filtering. The assessment asks to restructure these into JOINs and use appropriate indexes.

Optimized Query

```
SELECT s.Name,
    s.Marks,
    d.DeptName,
    d.HOD AS HeadOfDept
FROM Student s
INNER JOIN Department d
ON s.DeptID = d.DeptID
WHERE d.DeptName IN
    ('Computer Science', 'Electronics');
```

Explanation

The original query performs separate subqueries to obtain:

- Department name
- HOD
- Department filtering

These operations can be performed using a single JOIN.

The optimized query:

1. Joins Student and Department using DeptID.
2. Retrieves DeptName directly from Department.
3. Retrieves HOD directly from Department.
4. Filters departments using IN.

Indexes

Indexes can be created on columns frequently used for joins and filtering.

```
CREATE INDEX idx_student_dept
ON Student(DeptID);
CREATE INDEX idx_department_dept
ON Department(DeptID);
```

For department-name filtering:

```
CREATE INDEX idx_department_name
ON Department(DeptName);
```

Expected Result

The query returns students belonging to:

- Computer Science
- Electronics

along with their marks, department name and HOD.

Conclusion

Using a single JOIN instead of multiple correlated/nested subqueries simplifies the query and can improve execution efficiency. Indexes on join/filter columns can further assist query performance.

Q10. Debug the Relational Calculus Expression

Given Expression

The assessment gives:

The intention is to find students who scored more than **at least one** student in Department 2.

Error

The expression uses:

which means **for all students**.

But the requirement says:

More than **at least one** student in Department 2.

For "at least one", we need the **existential quantifier**:

Therefore, ∀ must be replaced with ∃.

Correct Relational Calculus Expression

Explanation

- Student(s) → s is a student.
- ∃t → there exists at least one student t.
- t.DeptID = 2 → t belongs to Department 2.
- s.Marks > t.Marks → student s has more marks than that student.

Department 2 Marks

From the given sample data:

Student	**Marks**
Deepak	60
Esha	55
Farhan	95

The corrected query asks whether another student has marks greater than **at least one** of these values.

For example, a student scoring 68 is greater than Esha's 55, so that student satisfies the condition.

Why the Original Gives an Incorrect Result

The original uses:

So it requires:

s.Marks>Deepak

AND

s.Marks>Esha

AND

s.Marks>Farhan

Since Farhan has 95 marks, a student must score more than 95 to satisfy the original condition.

But the requirement is only to score more than **one or more** Department 2 students.

Therefore, ∃ is correct.

Conclusion

The logical error is the use of the **universal quantifier ∀ instead of the existential quantifier ∃**.